Exact, independently checkable optimality receipts for BP-OSD on the [[144,12,12]] bivariate-bicycle code

Josh Philbrick[1, *]

[1] *Coherence Energy Labs*

(Dated: November 7, 2026)

General quantum decoding is NP-hard, which does not rule out checkable optimality witnesses on the instances that arise. We attach to each syndrome decoded by BP-OSD a dual solution of the Feldman–Wainwright–Karger linear-programming relaxation adapted to syndromes, verified in exact rational arithmetic from the check matrix, the weights and the syndrome alone, with no solver; when the relaxation is not tight, a tree of such duals over a partition of the feasible set closes the proof. On the [[144,12,12]] bivariate-bicycle code, in a preregistered campaign over 8 datasets from code-capacity to circuit-level noise, all 2,019 nontrivial syndromes received an exact receipt, 2,010 of them a single dual and 9 a branching tree; all 2,042 stored receipts replay against a pinned manifest, each accepted by two separately written checkers. A receipt fixes the optimum, so it scores a decoder without the true error; an exact solver trusted to optimality would give the same scores, and the receipt makes them checkable. In a second preregistered campaign, 6,500 fresh syndromes of the same datasets were decoded by BP-OSD at the min-sum scaling factor 1 of the ldpc library's decoder class and at 0.625, the default of its own Sinter wrapper. Scaling found significantly more optima on 4 datasets, each surviving a Bonferroni correction (not preregistered) over all 8 exact McNemar tests, for example 989 against 381 of 1,000 stress syndromes and 486 against 415 of 500 at circuit level; on the other 4 no difference was detected at this sample size (hit counts 2 to 4 apart, McNemar p at least 0.21). Receipts certify objective optimality only; tied optima can carry different logical frames. All results are simulations.

I. INTRODUCTION

Decoding a stabilizer code is NP-hard in general [1, 2], and belief propagation with ordered-statistics post-processing (BP-OSD) [3, 4] is a standard heuristic for quantum low-density parity-check codes, including the bivariate-bicycle codes proposed as near-term memories [5]. A BP-OSD implementation returns a correction without a witness of its quality, and the only property routinely checked, that the correction explains the syndrome, is satisfied equally by an optimal and a suboptimal answer.

Hardness does not preclude witnesses on particular instances. A companion manuscript, *Certifying quantum error-correction decoders*, released with this one (Sec. IX), attached linear-programming duals to matching decoders; those receipts proved optimality to within a small additive slack under floating-point weights. This paper asks whether the same discipline survives the move to a decoding objective that is not treated here as a single graphlike T-join problem (matching-based decoders for bivariate-bicycle codes do exist [6]; they solve a different problem than the one certified here), and demands a stronger receipt: an exact rational proof under a published integer objective.

We make three contributions. First, a receipt system for BP-OSD built from the Feldman–Wainwright–Karger (FWK) relaxation [7], checked in exact arithmetic by a checker that imports no producer code, over a written input domain that all seven of its implementations (Table I) enforce, with a certified repair when BP-OSD is refused and branch-dual trees when the relaxation is not tight. Second, a preregistered, frozen measurement of receipt coverage on the [[144,12,12]] code, from code-capacity to circuit-level noise under the code's published syndrome-extraction schedule. Third, the receipts as an instrument: because a receipt fixes the optimum, it scores a decoder's answers without the true error. Any exact solver run to proven optimality gives the same scores; what a receipt adds is that the optimum, and so every score, can be checked by a third party in exact arithmetic without trusting the solver. Certified per-shot benchmarking of decoders on this code is also the aim of concurrent work, for another objective (the outcome of non-degenerate maximum-likelihood decoding, judged against the sampled error) and with lower bounds that rest on a solver's floating-point arithmetic [8] (Sec. VIII). With it, a second preregistered campaign compares two settings of one BP-OSD parameter on fresh syndromes: one setting finds significantly more optima on half of the datasets, and on the other half no difference was detected at the campaign's sample size. Every number is a simulation; Sec. VII states what that does and does not limit.

II. PROBLEM AND RELAXATION

Given a sparse check matrix H over GF(2) with rows (checks) j of support $N(j)$, non-negative rational weights w, and a syndrome s, the decoding problem is to find $e \in \{0,1\}^n$ minimising $w^\top e$ subject to $He = s \pmod 2$. With $w_i = 1$ this is minimum Hamming weight. For heterogeneous and circuit-level noise we publish an integer

* Josh@coherenceenergylabs.com

weight vector, generated as $c_i = \lfloor 1000 \log((1-p_i)/p_i) + \frac{1}{2} \rfloor$, and that vector, not the logarithms it approximates, is the normative objective. On the 300 syndromes of the development heterogeneous dataset, the scale factors 10^3, 10^4 and 10^5 give exact receipts on 300 / 300 / 300 shots, with 0 changes in any selected correction; one shot at every scale needs a branch-dual tree. An earlier run of this check counted only flat receipts and left that shot to the solver's reported status, under which the shot changed tier at 10^5. Minimum weight under independent mechanism weights is not maximum likelihood over degenerate cosets, and every claim below is conformance to this stated specification.

Relaxation. Each check j is relaxed to its local codeword polytope [7], adapted to the syndrome bit: for every $F \subseteq N(j)$ with $|F| \not\equiv s_j \pmod 2$,

$$\sum_{i \in F} (1 - x_i) + \sum_{i \in N(j) \setminus F} x_i \;\geq\; 1. \tag{1}$$

Minimising $w^\top x$ over these constraints and $[0,1]^n$ lower-bounds the integer optimum, so any feasible dual solution certifies a lower bound.

Lemma 1 (safe dual)**.** *Write the validated facets as $a_r^\top x \geq b_r$. For any $y \geq 0$, let $z_i = \max\big(0, (A^\top y)_i - w_i\big)$. Then $L = \sum_r b_r y_r - \sum_i z_i$ is a lower bound on $w^\top e$ for every feasible e.*

Proof. For $x \in [0,1]^n$ satisfying the facets, $w^\top x \geq (A^\top y)^\top x - z^\top x \geq y^\top A x - \sum_i z_i \geq b^\top y - \sum_i z_i$. □

This is the safe bound of Neumaier and Shcherbina [9] for a linear program over bounded variables, here the box $[0,1]^n$: $b^\top y$ less the positive part of the residual $A^\top y - w$, weighted by the variables' upper bounds. They evaluate it with directed rounding, so that an approximate dual from a floating-point solver still gives a valid bound; the checker evaluates it in exact rationals. The lemma holds for every $y \geq 0$, so the checker needs no feasibility tolerance: it recomputes z and L exactly from whatever multipliers the certificate lists. If all weights lie in $(1/D)\mathbb{Z}$, every feasible objective lies on that lattice, and a lower bound $L > U - 1/D$ proves a feasible candidate of objective U optimal; $L = U$ is the special case of an attaining dual, and $L > U$ is an instrument failure, refused as one.

III. THE RECEIPT LADDER

A *receipt* is what a checker accepts or refuses for one syndrome: the instance (H, w, s), bound by hash, the candidate correction it proves optimal, if any, and a *certificate*, the proof itself (none is needed for $s = 0$): one dual of the relaxation, a flat certificate, or a branch-dual tree. A flat receipt is one whose certificate is flat. Every shot receives a receipt, and its tier is read from what the receipt proves.

Tier 0.: For $s = 0$ the empty correction is optimal at cost zero, because the checker refuses negative weights. This certifies the objective, not the absence of an undetected logical fault. Trivial shots are excluded from the nontrivial rates below.

Tier 1, BP-OSD's answer certified.: One dual of the shot's relaxation proves BP-OSD's own answer optimal: the checker verifies $He = s$, reconstructs every facet and cut, computes L exactly and verifies $L > U - 1/D$ for the answer's objective U.

Tier 2, certified repair.: One dual proves another correction optimal, and BP-OSD's answer is heavier or is not a correction; the replacement is the relaxation's integral primal or a solver candidate, and the checker re-verifies $He = s$ from scratch. In the instance that exposed this tier, BP-OSD returned weight 22 against a tight bound of 6; the difference was a weight-24 logical operator, and the certified repair lay in the same coset as the sampled fault.

Branching.: When no single dual closes, a mixed-integer solver proposes a candidate and the receipt is a tree: branching on a coordinate partitions the binary feasible set into $x_k = 0$ and $x_k = 1$, each again a syndrome instance, and every leaf carries either a safe dual proving the bound on the reduced instance or a GF(2) row combination showing that no completion exists. The checker walks the tree in exact arithmetic without a solver; nothing the solver reports is trusted. A branching tree proves the optimum's value, so a BP-OSD answer of that value is proven optimal by it too.

Redundant parity checks, sums of rows over GF(2), are generated adaptively in the manner of Zhang and Siegel [10]. Each cut is named by the rows it sums, and the checker re-derives its support and parity from (H, s).

The campaign's producer assigned tiers by its own route, not by this rule. It tried a single dual through a floating-point screen whose acceptance is exact equality, $L = U$, and stored every single dual that crossed only the lattice level as a one-node "tree". As a result every one of the 2,002 stored flat receipts attains $L = U$ exactly, and the lattice rule decides only the 8 single duals the producer filed as trees and the leaves of branching trees. Tables III and IX count tiers by the rule above, re-decided from the stored proofs by the production exact checker (`qldpc_tier_by_receipt.py`); the producer's labels are kept in Appendix A.

Two soundness statements are kept separate. The mathematical one: for inputs in the declared domain (Sec. IV), checker acceptance implies objective optimality, by the lemma and the partition argument. The implementation one: the checker is separately reimplemented, mutation-tested, and exercised on the released corpus. The second is strong evidence, not formal verification.

IV. THE CHECKER

The exact checker admits only rationals, rejecting floating-point values by type, and imports nothing beyond the Python standard library's `fractions`, `math`, `operator` and `dataclasses`; producers may use anything. The same module also carries a floating-point *screen* that the producer consults before spending an exactification on a candidate; no receipt is accepted on the screen's verdict, and every count in this paper is the exact checker's. Verification costs $O(\mathrm{nnz}(H_R)+\mathrm{nnz}(C)+n)$ arithmetic operations for the reconstructed redundant rows H_R and listed facet–multiplier pairs C, with bit complexity governed by the size of the rationals. Held-out certificates listed at most 91 facets, with numerators of at most 19 bits and denominators of at most 8 bits. Median exact checking time per shot ranged from 0.11 to 0.80 ms across datasets, against median producer times of 4.8 to 89.1 ms.

The input domain. Soundness is a statement about inputs in a declared domain, and the domain is written down (`docs/QLDPC_RECEIPT_INPUT_DOMAIN.md`): every index, check entry, branch variable and facet set is a non-boolean integer in range and every listed set is free of repeats; every syndrome entry is exactly 0 or 1; a combination of checks names at least two distinct rows; multipliers are non-negative rationals, written in JSON as integer numerator and denominator in lowest terms; every listed facet is validated, including one whose multiplier is 0, which adds nothing to the bound but is still part of the receipt; a root bound above the candidate's weight is refused. An integer is written as a JSON integer literal; an integer slot holding `1.0` is refused by every implementation, the JavaScript ones by reading the token itself, since a JavaScript number cannot tell it from `1`. All seven implementations reach one verdict on each of 109 shared vectors, which a test runs against all of them; how the contract came to be written is in Appendix A.

Separate implementations. Table I lists the seven. A reference checker for flat certificates was written from the mathematical specification of a draft of this paper, during an AI-assisted adversarial review of that draft that had no access to the repository, so without importing or inspecting the producer or the production checker; it is pinned by hash. A second tree checker was written in the project from the tree schema and the manuscript's branching tier, without importing or consulting the production checker. Third checkers, in JavaScript with exact rational arithmetic and no dependencies, were written from the manuscript's statement of the problem, in the same project, by the same author with AI assistance; the work that wrote them also wrote the harness that drives the production checker beside them. All of these are separately written, not independent: every one was written with AI assistance at one author's request, and agreement among them establishes agreement on the tested inputs, not the absence of a shared misreading of the specification. The JavaScript checkers accept all 2,002 flat receipts and refuse all 136 planted defects in 17 classes, each for its own reason, and accept all 40 stored trees and refuse all 74 planted tree defects in 19 classes.

TABLE I. The seven implementations of the input-domain contract (Sec. IV). P: one module, `src/oneq/qldpc_check.py`, written with the producers; R: `tools/external/`; JS: `verifier-js/`. Flat or tree is the form of receipt each checks; P-screen is the floating-point screen. Decides: "yes" for the two checkers of a receipt's form, both of which must accept every stored receipt, "third" for a further check of the stored corpus. Written from: as the files' headers and the private development repository's history record it. All seven were written with AI assistance, all but R-flat in the project. [a]During an AI-assisted adversarial review of a draft, without access to the repository.

name	language	decides	written from
P-flat	Python	yes	the derivation
P-tree	Python	yes	the derivation
P-screen	Python	no	the derivation
R-flat	Python	yes	a draft's specification[a]
R-tree	Python	yes	the tree schema and the manuscript
JS-flat	JavaScript	third	the manuscript's statement of the problem
JS-tree	JavaScript	third	the tree schema and the manuscript

Mutation testing. The checker's source is exercised by a hand-chosen set of targeted mutants, each of which a named test must detect: a sign inversion, a deleted feasibility pass, a syndrome taken on faith on each path that recomputes it, a forgotten parity on redundant checks, a loosened facet-parity rule, a removed slack derivation, a duplicate facet, and a lattice rule loosened by one level. Standard mutation operators applied at every eligible site of the checker module give 443 mutants; the tests kill 435, and each of the remaining 8 computes the same function as the original, for a reason recorded with the score. Certificate-level forgeries (a certificate replayed from another syndrome or matrix, a heavier feasible candidate, a floating-point multiplier, a dual inflated until its overload no longer proves the bound, an incomplete partition) are refused, and the second tree checker refuses all 5 of its forged-tree classes.

Bound-only results. The tree checker also runs without a candidate, proving that every solution of a system weighs more than a threshold T; frame determinacy (Sec. VII) and the distance bound of Sec. VI B use it. Its acceptance reports `LOWER_BOUND_PROVEN` with T and the lattice-rounded bound $(\lfloor TD \rfloor + 1)/D$, and no primal value or optimality claim.

V. ADAPTIVE FACET SEPARATION AT CIRCUIT-LEVEL DEGREES

Adaptive linear-programming decoding, which starts with no facets and repeatedly adds violated ones, is due to Taghavi and Siegel [11], who proved that at any point each check has at most one violated facet and that once none is violated the adaptive solution solves the full relaxation; they found the violated facet by sorting the check's coordinates. The rule used here needs no sorting and costs $O(\deg)$ per check: it is Algorithm 2 of Taghavi, Shokrollahi and Siegel [12], who note that Wadayama [13] independently proposed an equivalent test of membership in the relaxation. We restate it for syndromes. For a check j, write $\mathrm{LHS}(F)$ for the left side of (1) and $F_0 = \{i \in N(j) : x_i > \frac{1}{2}\}$; for every $F \subseteq N(j)$,

$$\mathrm{LHS}(F) = \mathrm{LHS}(F_0) + \sum_{i \in F \triangle F_0} \big|2x_i - 1\big|,$$

so the minimiser over the wrong-parity family is F_0 itself when $|F_0|$ already has the wrong parity, and otherwise F_0 with the coordinate of $N(j)$ nearest $\frac{1}{2}$ toggled. The shipped rule matches brute force over all wrong-parity subsets on 8,000 random instances with 0 suboptimal separations (largest discrepancy 1.8×10^{-15}), and agrees with a separately written reference implementation on 20,000 further instances, including exact ties at $\frac{1}{2}$. At convergence no facet of the full family is violated, so the lazy relaxation equals full enumeration; on 40 random codes where enumeration was feasible the two agreed to within 4.5×10^{-16}.

This matters at circuit level, where detector degrees reach 47 and a full facet family has 2^{46} rows. Degree-capped enumeration remains sound, since a subset of facets only enlarges the feasible region, and matched full tightness on 23/40 random codes; adaptive separation needs no cap. Deployed first, with enumeration only on refusal, it gave identical coverage on a circuit-level surface-code benchmark at 111.15 against 1183.87 ms per shot, a factor of 10.7.

VI. RESULTS

Two sampling paths, both on a single workstation: the code-capacity, heterogeneous, phenomenological and X-sector datasets draw independent faults with NumPy's generator under the stated rates; the circuit-level dataset samples its detector error model with Stim [14]. Table II states each dataset's check matrix, size, fault model and decoder prior. The "uniform" and "X-sector" datasets at $p = 0.02$ differ only in the matrix: the first decodes X faults with the Z checks H_Z, the second Z faults with H_X, each a 144-column single-sector problem. BP-OSD (`ldpc` 2.4.1) used min-sum belief propagation with a parallel schedule, 30 iterations (40 at circuit level), and OSD-CS of order 5 [4]. The min-sum scaling factor is 1 by default in the library's `BpOsdDecoder` class (`ldpc/bposd_decoder/_bposd_decoder.pyx`, line 53) and 0.625 by default in its Sinter wrapper around that class (`ldpc/sinter_decoders/sinter_bposd_decoder.py`, line 41, whose docstring says 1.0); the held-out campaign of Sec. VI A used the class default, 1, and Sec. VI C compares the two.

A. Preregistered held-out campaign

The protocol, datasets, endpoints and seed rule (`docs/HELD_OUT_PREDECLARATION.md`) were committed and tagged in the author's development repository before any held-out shot was sampled. That repository and its history are private, and its commit dates are set by its author, so the order rests on the author's record; the history is available to referees on request. What is public is the predeclaration as shipped, the freeze commit's identifier with the SHA-256 of the predeclaration at the freeze (`evidence/freeze_identity.json`), the per-shot records, and a snapshot of the frozen code (Sec. IX); the shipped predeclaration differs from the frozen text by two later notes, a corrected description of the seed source and a pointer to the results. Each dataset's seed is the first 16 hexadecimal digits of SHA256(`"oneq-heldout-2026||"` + dataset + `"||"` + freeze commit), masked to 63 bits and reduced modulo $2^{31} - 1$ for the circuit dataset; all seeds re-derive from the freeze commit's identifier by this rule. The rule makes the samples replayable; it is not an external randomness beacon, and local evidence cannot exclude a search over alternative freeze states. In the development repository the `src/` and `experiments/` directories are byte-identical between the freeze and results commits, and no shots were excluded or re-run. Sampling continued until the predeclared number of nontrivial syndromes was reached, except on the circuit dataset, which used 150 sampled shots of which one was trivial (a recorded deviation, Appendix A). Table III gives the result.

All 2,019 nontrivial syndromes received exact receipts: 2,010 (99.55%) a single dual, 2,002 of them attaining $L = U$ and 8 closing by the lattice rule, and 9 a branching tree. BP-OSD's own answer is proven optimal on 1,783 shots: 1,780 in tier 1 and 3 by branching trees. On the other 236 the receipt proves BP-OSD's answer strictly heavier than the optimum: 230 certified repairs by a single dual and the rest by branching trees. That is a property of the decoder as the campaign configured it, and it is concentrated on one dataset: 178 of those answers are on the stress dataset. The per-dataset bounds in Table III concern certification coverage, not logical correctness, and are not a simultaneous guarantee. No receipt in these totals rests on a solver's reported status. Across held-out and development datasets, the 40 stored trees had a median of 3 nodes, a 95th percentile of 21, at most 25 nodes and depth 7, and at most 20,505 serialised bytes. Tree size is exponential in the worst case, and this is not a polynomial-time exact decoder.

TABLE II. The datasets. Rows and columns of the check matrix as persisted (hash-bound) in the release; the phenomenological matrix stacks four noisy rounds and one perfect round (576 data and 288 measurement mechanisms), the circuit one is the detector error model of the published depth-8 cycle over three rounds of memory-Z with one declared observable. Prior: the decoder's channel. Objective: unit weights, or the published integer weights.

dataset	check matrix	rows × columns	fault model	prior	objective
uniform, $p = 0.01$	H_Z	72 × 144	i.i.d. X	p	unit
uniform, $p = 0.02$	H_Z	72 × 144	i.i.d. X	p	unit
heterogeneous, $p = 0.02$	H_Z	72 × 144	independent X, $p_i \sim 0.02\,U(\frac{1}{2}, 2)$	p_i	integer
phenomenological, $p = p_m = 0.01$	spacetime H_Z	360 × 864	i.i.d. data and measurement	p	integer
phenomenological, $p = p_m = 0.02$	spacetime H_Z	360 × 864	i.i.d. data and measurement	p	integer
X-sector, $p = 0.02$	H_X	72 × 144	i.i.d. Z	p	unit
stress, $p = 0.05$	H_Z	72 × 144	i.i.d. X	p	unit
circuit, reference schedule	DEM	288 × 2,304	circuit-level, idles unfaulted	DEM	integer

TABLE III. Preregistered held-out campaign on the [[144,12,12]] code. N counts nontrivial syndromes. Tiers are read from the receipt (Sec. III): tier 1 is BP-OSD's own answer proven optimal by one dual, tier 2 a replacement proven optimal by one dual, "branching" a tree with at least one branch. The last column is the one-sided exact 95% Clopper–Pearson lower bound on the certified fraction, floored. Phenomenological datasets use four rounds of noisy syndrome measurement.

dataset	sampled	tier 0	N	tier 1	tier 2	branching	certified	95% lower
uniform, $p = 0.01$	398	98	300	300	0	0	300	$\geq 99.00\%$
uniform, $p = 0.02$	493	23	470	450	20	0	470	$\geq 99.36\%$
heterogeneous, $p = 0.02$	306	6	300	299	1	0	300	$\geq 99.00\%$
phenomenological, $p = p_m = 0.01$	150	0	150	150	0	0	150	$\geq 98.02\%$
phenomenological, $p = p_m = 0.02$	150	0	150	149	0	1	150	$\geq 98.02\%$
X-sector, $p = 0.02$	206	6	200	194	6	0	200	$\geq 98.51\%$
stress, $p = 0.05$	300	0	300	122	172	6	300	$\geq 99.00\%$
circuit, reference schedule	150	1	149	116	31	2	149	$\geq 98.00\%$

A post-hoc re-decode (Table IV; `qldpc_heldout_scaled_posthoc_all8.py`; not preregistered) regenerates every dataset's decoder from the campaign's own settings, first reproduces every stored BP-OSD answer at scaling 1.0, and then decodes each syndrome at 0.625, scoring both against the stored receipts. Over the 2,019 syndromes the optimum hits move from 1,783 to 2,002 (equal to the number of flat receipts by a coincidence of totals; dataset by dataset the two differ), but on 3 datasets the scaled decoder finds fewer, and on several each setting finds optima the other misses. Because those syndromes were known when the question was asked, Sec. VI C asks it again on fresh ones.

The complete stored corpus, 2,042 receipts (the 2,002 held-out flat receipts, the 17 held-out trees and 23 development trees), replays against a manifest whose hash is pinned in the replay tool: every receipt is checked by two separately written checkers (Sec. IV) against persisted, hash-bound check matrices and integer weight vectors, and the replay fails if any receipt or bound file is missing, duplicated, unlisted or altered. The persisted instances match the instance hashes recorded by the frozen run, and the complete corpus replays under the checkers of the input-domain contract.

On the stress dataset the predeclared paired comparison of logical-frame outcomes, judged retrospectively against the sampled fault, was: both correct 125; BP-OSD wrong and receipt pipeline correct 170; BP-OSD correct and pipeline wrong 0; both wrong 5 (exact two-sided McNemar $p = 1.3 \times 10^{-51}$, a descriptive summary of this dataset against the stated configuration only). This measures the configuration, not BP-OSD: re-decoded post hoc with scaling 0.625, BP-OSD commits 8 logical errors on these 300 syndromes against the receipt pipeline's 5, wrong where the pipeline is right on 3 and the reverse on 0 (exact two-sided McNemar $p = 0.25$), which establishes no logical advantage over the scaled decoder.

B. Circuit level under the published schedule

The circuit dataset implements the code's published depth-8 syndrome-extraction cycle [5] in Stim, memory-Z over three rounds at uniform circuit-level noise $p = 0.002$, with 288 detectors and 2,304 mechanisms under published integer weights. The noise family is ours and, unlike the original error model, does not fault idle locations. At three rounds this leaves the set of distinct fault signatures unchanged, which is what the distance comparison below needs: aligned by the measurement events each detector compares, the model's 2,304 mechanisms (distinct detector and observable signatures) are exactly the 2,304 of the companion manuscript's model of the same cycle, whose noise

TABLE IV. Post hoc, on the held-out syndromes of Table III: optimum hits of BP-OSD at the campaign's scaling factor 1.0 and at 0.625, scored against the stored receipts, the shots on which only one setting finds the optimum, and the exact two-sided McNemar p of that split. At 1.0 the decoder reproduces every stored answer.

dataset	N	hit, 1.0	hit, 0.625	1.0 only	0.625 only	McNemar p
uniform, $p = 0.01$	300	300	300	0	0	1
uniform, $p = 0.02$	470	450	468	2	20	1.2×10^{-4}
heterogeneous, $p = 0.02$	300	299	298	1	0	1
phenomenological, $p = p_m = 0.01$	150	150	148	2	0	0.5
phenomenological, $p = p_m = 0.02$	150	150	149	1	0	1
X-sector, $p = 0.02$	200	194	199	0	5	0.0625
stress, $p = 0.05$	300	122	299	0	177	1.0×10^{-53}
circuit, reference schedule	149	118	141	3	26	1.5×10^{-5}

does include idle locations, projected to its Z-type detectors (`qldpc_refsched_p3_equality.py`); the idle faults add no signature. Equal signature sets do not establish equal mechanism probabilities, likelihood weights or logical error rates for the two models, and no result here compares them. The transcription was validated at the fault level, not only noiselessly: an operation generator written separately from a hashed layer manifest reproduces the circuit exactly (325 operations, 576 measurement records), and every single X and Z fault at every location, 12,816 in total, was propagated by a separately written symplectic engine and compared with Stim's detector and observable signature, with 0 mismatches. The held-out run certified all 149 nontrivial shots. The circuit declares one logical observable, and it agreed with the sampled truth on 149 of 149 shots; this is a result for that observable, not an all-12-logical-qubit block error, and zero observed disagreements in 149 shots bounds the underlying disagreement rate below $1-0.05^{1/149} \leq 2.00\%$ (one-sided 95%). BP-OSD's own answers, scored the same way, also agreed on 149 of 149: the receipts replaced 31 of them with a strictly lighter correction, and all 31 replacements lie in the same logical class as the answer they replaced, so at this noise level the receipt changed the weight of the answer and never its outcome (`tools/bposd_circuit_outcome.py`).

Two further circuit datasets, a surface-code control and a custom sequential schedule, were checked only in floating point. Floating-point acceptance is not a receipt in the sense of this paper, and they are not counted here.

The distance of the declared observable. Let d_{DEM} be the weight of the lightest undetectable fault that flips the circuit dataset's declared observable. A 10-column fault in the persisted instance, the companion manuscript's two-round witness shifted by one round, fires no detector and flips the observable, and no nine of its columns do both, so $d_{\mathrm{DEM}} \leq 10$; Stim's heuristic search had found only weight 12. Because the mechanism sets are equal, and the declared observable is, mechanism by mechanism, the first of the companion model's twelve, so that a fault flipping it flips one of them, the companion manuscript's certified lower bound for this cycle at three rounds applies to this model unchanged; over its twelve observables it is 10, so $d_{\mathrm{DEM}} = 10$ for the declared observable at three rounds. That is derived from the companion's certificate and its time-shift theorem; only three rounds were compared. The companion, *Machine-checkable lower-bound certificates for the circuit-level distances of bivariate-bicycle syndrome-extraction circuits*, establishes circuit distance 10 for every number of rounds from two up under its own fault placement. Each under its own noise model, the same value was found by an exhaustive search for twelve noisy rounds [15] and by exact enumeration for up to three rounds [16], neither of which records a certificate. Guancheng Pan's later research records certify the lower bound with VeriPB proofs for up to twelve rounds [17] and argue it for every number of rounds: under SD6 circuit noise from solver runs without certificates [18], and for his re-implementation of IBM's fault model from those certificates and a written argument without a certificate for the step [19]. Bin Cheng, Naixu Guo and Feng Pan [20] certify 10 in both bases at fixed numbers of rounds (one noisy cycle of the memory instruction of IBM's architecture study, twelve noisy cycles of the circuits of IBM's public code) with an exact solver whose completeness they prove, pairing its lower bound with a logical error of matching weight; their records carry no proof object for the lower bound. This paper's own machinery did not reach the bound: the flat relaxation of the distance problem gives only 1.0, and tree closure at the tight threshold did not finish within 200,000 nodes. Certifying distance lower bounds is where branch-and-bound is expected to be expensive; machine-checked code distances are the object of Ehatamm *et al.* [21], and nothing else in this paper depends on a distance claim.

C. Preregistered comparison of two scaling settings

Before any of its syndromes was sampled, a second campaign was committed to the same private development repository (`docs/PREREG_qldpc_scaling_2026-11-03.md`, shipped unchanged since that commit; as for the first, the order rests on the author's record) with its runner, fresh seeds stated in full, the

eight dataset families of Table II, 6,500 nontrivial syndromes in all, and its analysis. Each dataset's seed is derived as in Sec. VI A, from the SHA256 of the domain string `oneq-qldpc-scaling-prereg-2026-11-03`, the separator `||` and the dataset name, with no commit in the hash; as there, the rule makes the samples replayable but is not an external randomness beacon, and the domain string was the author's choice. Every syndrome is decoded twice, at scaling factor 1.0 and at 0.625, all other parameters equal, and receives one exact receipt for its optimum, checked by both of its checkers; the optimum does not depend on either answer. Each answer is then scored against the receipt. The primary comparison is, per dataset, the exact two-sided McNemar test of optimum hit on the same syndromes; the eight tests are reported uncorrected and descriptively, and no pooled comparison is made. One deviation is recorded (`docs/PREREG_qldpc_scaling_2026-11-03_DEVIATIONS.md`). The runner's six-round redundant-parity LP took 18.6, 42.9 and 206.5 s on three of the first 81 syndromes of the phenomenological dataset at $p = 0.02$, and that dataset did not finish in the 3,000 s allowed, so the runner was changed to give the phenomenological datasets the lazy-separation LP the circuit dataset already used. The phenomenological dataset at $p = 0.01$ had already run as committed, so the change affected one dataset, the one at $p = 0.02$ (500 of its 500 receipts flat). The change alters the form of a receipt, not the certified optimum it proves, and so no score. Table V gives the result. Every one of the 6,500 syndromes received a receipt (unresolved: 0).

The scaled setting finds significantly more optima on 4 of the 8 datasets, and they lie where the default's answers are worst: on the stress dataset 989 optima against 381 of 1,000 (McNemar $p = 1.1 \times 10^{-177}$), on the X-sector dataset 995 against 936 ($p = 3.5 \times 10^{-18}$), on the uniform dataset at $p = 0.02$, 994 against 954 ($p = 4.6 \times 10^{-10}$), and at circuit level 486 against 415 of 500 ($p = 8.4 \times 10^{-17}$). Each of these p is at most 5×10^{-10}, below 0.05/8, so all 4 survive a Bonferroni correction over the 8 tests; the correction is not in the preregistration and is applied here only to qualify the uncorrected tests. On the other 4 datasets no difference was detected at this sample size: their hit counts are 2 to 4 syndromes apart, with $p \geq 0.21$. That is an absence of evidence for a difference, not evidence of equivalence; no equivalence test was run. As point estimates only, the scaled setting finds 999 against 995 on the uniform dataset at $p = 0.01$ (split 1 to 5, $p = 0.219$), and two fewer on the heterogeneous dataset and on each phenomenological dataset. Every dataset has syndromes on which only one setting finds the optimum. The hits proven by a branching tree rather than a single dual are few (Table V): 0 and 16 on the stress dataset, 8 and 8 at circuit level, none elsewhere; every miss is a heavier correction. Judged against the sampled fault (Table VI), the scaled setting commits fewer logical errors on all 5 code-capacity datasets, significantly on 3 ($p \leq 2 \times 10^{-12}$): the uniform dataset at $p = 0.02$, 0 against 40; the X-sector dataset, 3 against 58; and the stress dataset, 31 against 595, where the receipt's own answers commit 26. On the uniform dataset at $p = 0.01$ (1 against 3, $p = 0.625$) and the heterogeneous one (0 against 1, $p = 1$) the difference is not significant. At circuit level the declared observable agrees with the sampled truth on 500 and 500 of 500 at the two settings, so there the default's heavier answers changed the weight, never the outcome. The size of the effect depends on the dataset; this does not rank the two settings in general, and it says nothing about maximum likelihood or about the block's twelve logical qubits.

Costs. The run of record timed whole datasets only, and its per-shot production times were not recorded. We therefore re-ran the campaign's own runner on its preregistered seeds, each dataset under the bytes it ran under (runner, producer and all three checker modules, recovered from the development repository's history by the hashes in its summary), with a timer around every stage of every shot (`qldpc_scaling_prereg_timed_rerun.py`). The re-run reproduced all 6,500 records byte for byte, receipts included, with 0 unresolved shots, so its costs are those of exactly the receipts reported. Table VII separates producing a receipt (the linear program, its rationalisation, the production checker's acceptance of a candidate and, where no single dual closes, the MILP candidate and the branch-dual tree) from checking the stored receipt with both of its checkers, timed by the replay of Sec. IX on the same receipts. Production medians range from 8.2 to 140 ms per syndrome and checking medians from 2.5 to 127 ms, most of it the second checker's on every dataset (the reference flat checker takes the check matrix as a dense array); the production checker alone has medians of 0.311 to 4.9 ms. The tails are long, which medians would hide: the slowest receipt, a branch-dual tree of 3 nodes on the circuit dataset, took 607.8 s to produce, 99% of it in the branch-dual search over its node relaxations, while the slowest check took 156 ms, on the circuit dataset. Decoding is not in either column: both settings together took a median of at most 0.772 ms per syndrome on any dataset. The MILP was called on 28 syndromes, each call returning HiGHS's optimal status inside its 30 s limit, and all 28 trees closed inside their caps of 60,000 nodes and depth 40; the second tree checker had 600 s per call, and nothing had a memory cap. All of it ran on one workstation (Intel Core Ultra 9 275HX, 24 logical CPUs), one process per dataset in three partly concurrent lanes, on a machine shared with other work and 37 to 71% busy overall; each run held up to 64 operating-system threads (library thread pools) but used about one core (CPU over wall time 0.98 to 1.00), and at most 353 MB of memory. These are measurements of one machine under that load, not a benchmark across hardware.

TABLE V. The preregistered comparison: fresh syndromes, BP-OSD at min-sum scaling 1.0 (the class default) and 0.625 (the Sinter wrapper's default), every other setting equal. Hit: the answer is a correction whose integer objective equals the certified optimum; in parentheses, the hits proven by a branching tree, the rest being accepted by the syndrome's single dual. Every miss is a heavier correction (answers that are not a correction: 0). "1.0 only", "0.625 only": shots on which only that setting hits. McNemar: exact two-sided, uncorrected, one dataset at a time.

dataset	N	unresolved	hit (tree), 1.0	hit (tree), 0.625	1.0 only	0.625 only	McNemar p
uniform, $p = 0.01$	1,000	0	995 (0)	999 (0)	1	5	0.219
uniform, $p = 0.02$	1,000	0	954 (0)	994 (0)	3	43	4.6×10^{-10}
heterogeneous, $p = 0.02$	1,000	0	996 (0)	994 (0)	4	2	0.688
phenomenological, $p = p_m = 0.01$	500	0	500 (0)	498 (0)	2	0	0.5
phenomenological, $p = p_m = 0.02$	500	0	496 (0)	494 (0)	5	3	0.727
X-sector, $p = 0.02$	1,000	0	936 (0)	995 (0)	0	59	3.5×10^{-18}
stress, $p = 0.05$	1,000	0	381 (0)	989 (16)	3	611	1.1×10^{-177}
circuit, reference schedule	500	0	415 (8)	486 (8)	6	77	8.4×10^{-17}

TABLE VI. Secondary, preregistered, code-capacity datasets of Table V: logical errors against the sampled fault for each setting, the shots on which only that setting errs, the exact two-sided McNemar p of that split (uncorrected), and the logical errors of the receipt's own answer.

dataset	errors, 1.0	errors, 0.625	1.0 only	0.625 only	McNemar p	receipt answer
uniform, $p = 0.01$	3	1	3	1	0.625	0
uniform, $p = 0.02$	40	0	40	0	1.8×10^{-12}	0
heterogeneous, $p = 0.02$	1	0	1	0	1	0
X-sector, $p = 0.02$	58	3	56	1	8.0×10^{-16}	1
stress, $p = 0.05$	595	31	564	0	3.3×10^{-170}	26

TABLE VII. Stage-separated cost of the preregistered campaign's receipts, per nontrivial syndrome: nearest-rank median, 90th and 99th percentiles and maximum over the dataset's N syndromes. Production: producing the receipt (linear program, rationalisation, the production checker's acceptance, and the MILP and branch-dual tree where needed), timed in a re-run that reproduced every record byte for byte; decoding, the second checker and scoring are excluded. Checking: both checkers on the stored receipt, timed by the replay of Sec. IX. One workstation under shared load (Sec. VI C).

		production (ms)				checking, both checkers (ms)			
dataset	N	median	p90	p99	max	median	p90	p99	max
uniform, $p = 0.01$	1,000	8.2	10	12	381	2.5	3.0	3.3	3.7
uniform, $p = 0.02$	1,000	8.5	11	120	2,164	2.5	3.0	3.2	4.1
heterogeneous, $p = 0.02$	1,000	9.4	11	628	14,239	2.6	2.8	3.3	3.8
phenomenological, $p = p_m = 0.01$	500	140	152	180	2,764	55	66	70	73
phenomenological, $p = p_m = 0.02$	500	17	30	10,064	51,839	56	64	71	73
X-sector, $p = 0.02$	1,000	10	12	14	3,800	2.5	2.6	2.7	4.0
stress, $p = 0.05$	1,000	9.1	1,349	40,176	162,801	2.7	3.5	15	82
circuit, reference schedule	500	27	39	17,367	607,826	127	148	153	156

TABLE VIII. Logical-frame errors, judged retrospectively against the sampled fault, on 200 shared nontrivial syndromes per noise level, for the fixed certified reference, for the receipt pipeline run under configurations A / B / C, and for BP-OSD itself under those and the post-hoc scaled configurations A_s / B_s. The last two columns count, under configuration C, the shots on which the pipeline kept a different optimal correction from the reference, and those on which that optimum lies in a different logical frame. †As run, at declared rate p: the unscaled answers depend on the floating-point form of the declared rate, A's counts slightly and B's so much that no comparison is drawn from them beyond the one in Sec. VI D.

	fixed	receipt pipeline	BP-OSD	C: pipeline vs reference	
p	reference	$A^\dagger$ / $B^\dagger$ / C	$A^\dagger$ / $B^\dagger$ / C / A_s / B_s	other optimum	other frame
0.01	0	0 / 0 / 0	1 / 0 / 0 / 0 / 0	0	0
0.02	0	0 / 0 / 0	9 / 0 / 3 / 1 / 1	1	0
0.03	0	0 / 0 / 0	31 / 1 / 0 / 0 / 0	1	0
0.05	9	9 / 9 / 8	114 / 109 / 9 / 7 / 9	10	2
0.07	34	34 / 34 / 36	172 / 41 / 32 / 35 / 36	16	2

TABLE IX. The instrument, two ways, on the same 200 shared syndromes per noise level and configurations A / B / C, and A_s / B_s (scaling factor 0.625, post hoc). Tier 1: BP-OSD's own answer proven optimal by one dual (Sec. III), re-decided from the stored proofs. Optimum hit: BP-OSD's answer feasible with weight equal to the certified optimum, whether a single dual or a branching tree proves it; every tier-1 answer is an optimum hit. For the unscaled A and B the optimum-hit entry is as run (declared rate p), followed in brackets by its range over 16 declared rates that exact arithmetic cannot tell apart. †As run, at declared rate p: A's entries vary little with the floating-point form of the declared rate, B's so much that no comparison is drawn from them beyond the one in Sec. VI D.

	tier 1 (one dual)					optimum hit				
p	A†	B†	C	A_s	B_s	A†	B†	C	A_s	B_s
0.01	199	200	200	199	199	199 [199–200]	200 [198–200]	200	199	199
0.02	190	200	197	196	197	190 [183–190]	200 [182–200]	197	196	197
0.03	165	199	200	199	198	165 [162–165]	199 [165–200]	200	199	198
0.05	84	87	196	195	195	84 [84–88]	87 [87–197]	198	197	197
0.07	24	163	179	178	178	24 [21–24]	171 [20–178]	196	196	196

D. A strong baseline, and what the configuration explains

We also reran 200 shared nontrivial syndromes per noise level under three configurations: A, the campaign's; B, min-sum with 100 iterations and OSD-CS of order 10; and C, product-sum with 100 iterations and OSD-CS of order 10. A and B use the class default scaling factor of 1. After these results were known we added, post hoc, A_s and B_s: A and B with the scaling factor 0.625. It was not tuned on these data, and every factor we tried below 1 gives nearly the same result. Their answers are classified by the same stored proofs and the same rule (`qldpc_strong_baseline_scaled.py`, which replays from its records and the proofs alone). Every reference answer and every receipt-pipeline answer counted in this experiment carries a checked exact receipt, flat or tree; a shot without one would be reported unresolved, and none was. BP-OSD's own raw answers, also counted, carry none. Every proof is retained, 891 flat certificates and 64 trees, content-addressed, and every decision names its proof: `tools/replay_strong_baseline.py` rebuilds every number of Tables VIII and IX from the records and proofs alone, 996 proof checks over 4,000 decisions with no producer, no solver and no BP-OSD. A check is one proof re-decided for one answer: a flat certificate is the syndrome's dual and proves every answer of the optimal weight, so a proof shared by two different answers is checked for each, and decisions sharing both a proof and an answer are checked once. The largest tree has 57 nodes and 29,905 serialised bytes (a median node count of 1 at $p = 0.07$). The fixed reference is independent of BP-OSD: the relaxation's certified primal where there is one, otherwise a solver candidate kept only if a branch-dual tree for it is accepted (12 and 52 shots at $p = 0.05$ and 0.07). The receipt pipeline is run separately under each configuration, keeping BP-OSD's answer whenever it is certified.

Three observations follow. First, the receipt pipeline's error count depends on the BP-OSD configuration even though every answer it returns is a certified minimum-weight correction: optimality does not fix the logical frame. Under configuration C the pipeline kept a different optimum from the reference on 10 shots at $p = 0.05$ and 16 at $p = 0.07$, with a different frame on 2 and 2 of them, and committed 8 and 36 errors where the reference committed 9 and 34. Second, the large advantages at stress noise are advantages over an unscaled min-sum decoder, not over BP-OSD as such: at $p = 0.05$ configuration C commits 9 errors, the same as the fixed reference, and at $p = 0.07$ it commits 32, fewer than either certified arm; and scaling alone repairs A, which at $p = 0.05$ commits 108–114 errors unscaled, whatever error rate it is told, and 7 scaled.

On this degenerate code an unscaled decoder's answers also depend on the error rate it is told, which in exact arithmetic they cannot: with a uniform prior every min-sum update, the hard decision and the OSD ordering are homogeneous in the prior log-likelihood ratio, and declared rates whose prior log-likelihood ratios are 1, 2, 4 and 8, which floating-point arithmetic scales exactly, give identical answers on all 200 shots. Other declared rates differ only in how that ratio rounds, yet the unscaled answers change with them: across the 16 declared rates we scanned, only 88 (A) and 85 (B) of the 200 shots at $p = 0.05$ keep one answer. For A this barely moves the counts (the ranges in Table IX); B finds the optimum on 87–197 of the $p = 0.05$ syndromes and 20–178 of the $p = 0.07$ ones. B at scaling 1 is therefore not a well-defined arm. Its entries in Tables VIII and IX are as run, at declared rate p, and we draw no comparison from them, except that at $p = 0.07$ B_s finds more optima (196) than B does at any declared rate. Scaled, at $p = 0.05$, A_s and B_s fail 7 and 9 times at every declared rate, and B fails between 9 and 11 times at each scaling factor we tried below 1 (0.9, 0.75, 0.625; `qldpc_bposd_scaling_sensitivity.py`).

Much of what goes wrong at scaling 1 is exact cancellation in the soft outputs. On the $p = 0.05$ syndromes BP converges on only 81 of 200 for A, and on 125 of the 200 some soft output ends exactly at zero, which never happens scaled. Breaking those ties removes most of the difference in optimum hits, in the spirit of the random prior perturbation Poulin and Chung use to break the symmetry of degenerate errors [22]. Concurrently, Fregona *et al.* [23] prove that Tanner-graph symmetries they call reflections keep parallel product-sum BP from converging on some weight-3 errors of bivariate-bicycle codes including this one, report a numerical instability in the BP-OSD implementation of Ref. [4] (infinite messages of opposite sign combining to NaN), and repair both by modifying BP, one variant with a bounded random perturbation. A fixed relative perturbation of the prior log-likelihood ratios by at most 10^{-6} keeps the exact-arithmetic invariance and leaves no exact zero. It raises A's convergence to 133, short of the scaled decoder's 172, and makes A's answers independent of the declared rate on all 200 shots. It raises A's optimum hits to 185 at $p = 0.05$, 175 at $p = 0.07$ and 285 of 300 on the held-out stress syndromes, at least 87% of the way from the unscaled to the scaled decoder (`qldpc_minsum_rate_scan.py`, `qldpc_minsum_rate_invariance.py`; both post hoc). Minimum weight is not maximum likelihood over degenerate cosets, and these counts are samples, not a performance floor.

Third, what survives is an instrument, and it needs two numbers kept apart (Table IX). The *tier-1 count* is how often one dual proves BP-OSD's own answer optimal: from 84 under A to 196 under C of 200 at $p = 0.05$, and from 24 to 179 at $p = 0.07$. Because every shot carries a certified optimum U_j, a stronger measurement is available without the true error: BP-OSD found an optimum exactly when its answer is feasible with weight U_j, whether a single dual or a branching tree proves U_j. That *optimum-hit rate* is 42.0% under A and 99.0% under C at $p = 0.05$, and 12.0% and 98.0% at $p = 0.07$; scaled,

A_s reaches 98.5% and 98.0%. The difference between the two columns is the optimal answers whose proof needs branching, a property of the relaxation on that syndrome rather than of the decoder, so the optimum-hit rate is the measure of decoder quality.

E. Development data and mechanism validation

Before the held-out campaign, development datasets over the same code covered the same noise families. Their 1,851 flat certificates were checked by the reference checker when produced and were not stored, so that check cannot be replayed. A later re-run of the same producer reproduced every development per-shot record except the certificate's hash, which differed on 70 shots (another dual proving the same answer optimal); its 1,851 certificates are stored (`evidence/p2_certificates.jsonl.gz`), 1,781 of them with the hash the original run recorded, and a checker written in Lean, whose acceptance is proven there to imply optimality (the proof covers the checking function, not its JSON reader or the compiler), accepts all 1,851 and refuses all 11,106 forgeries made from them (`tools/check_lean_certificate.py`, which needs Lean and Mathlib). Every one of the 23 shots the flat relaxation left open was closed by a tree, all 23 of which are stored and replayed. The held-out campaign supersedes those rates. Each mechanism was first validated against exact ground truth on random codes: on 60 codes the base relaxation was tight on 43, redundant-parity cuts raised that to 59, and no bound ever exceeded a feasible answer's weight (0 cases). On stress-noise samples the fractional residue was 6 shots of 120 after 6 rounds of cut generation and 6 after 25, a measured plateau whose structure we leave open.

VII. SCOPE AND LIMITATIONS

Every shot is simulated. No device has run the [[144,12,12]] code, so every rate is a property of the code, the noise model and the decoder configuration. Soundness does not depend on where a syndrome came from: the checker validates (H, w, s) alone, and a syndrome from a device is the same object. What hardware would change is the certification rate.

Conformance, not maximum likelihood. A receipt proves minimum weight under the declared objective. It does not prove that the sampled fault was recovered, that the optimum is unique, or that all optima share a logical frame. Frame determinacy can itself be proven with the tree machinery in bound-only mode, one tree per logical bit on the system augmented by that bit: on 300 random instances this gave 231 proofs, all correct, 69 refusals and 0 false proofs, and 5 of 5 demonstration shots of the code were proven determinate on all 12 logical bits in 0.8–2.2 s each. The tables above do not carry frame receipts.

Other limits. One decoder implementation; one qLDPC code; the stress comparison is not compute-matched; shot counts are limited by one workstation. Every certificate is shipped rather than left to be regenerated, because regenerating them does not reproduce every one byte for byte (Appendix A).

VIII. RELATED WORK

Certifying algorithms [24] and program checking [25] are classical. Linear-programming decoding and the property that an integral optimum certifies maximum likelihood are due to Feldman, Wainwright and Karger [7]; adaptive linear-programming decoding to Taghavi and Siegel [11], and the separation rule used here, which needs no sorting, to Taghavi, Shokrollahi and Siegel [12] and, independently, Wadayama [13]; adaptive cut generation to Zhang and Siegel [10]. Linear-programming decoding has been extended to quantum stabilizer codes [26]; local-optimality certificates exist for Tanner codes [27]; integer programming has been used to decode quantum codes, with optimality living inside the solver [28], and such a solver run to optimality would score a decoder exactly as a receipt does, but without a proof a third party can check; for classical codes, Draper, Yedidia and Wang [29] made adaptive linear-programming decoding a maximum-likelihood decoder by requiring the least certain bit of each fractional solution to be integral, which the branching tier here does by branching, and left the resulting mixed-integer program, and so the optimality, to an off-the-shelf solver; and linear programming has been combined with message passing and with OSD for quantum LDPC codes [30, 31]. Wu *et al.* [32] decode by a primal–dual minimum-weight parity factor algorithm that emits a proximity bound with every solution, and a dual meeting the primal's weight certifies optimality. That is the form of every stored flat receipt here, all of which attain $L = U$; the lattice rule closes only the single duals the producer stored as one-node trees and the leaves of branching trees. On classical block codes, Hwang, Won and Hwang [33] certify per frame, by a linear-programming dual bound or a distance test, that a neural decoder's candidate is a global maximum-likelihood solution; their dual test runs in floating point with a margin, so that a frame it certifies is "ε-optimal in likelihood cost rather than exactly optimal". What differs is where the certificate lives and who checks it: here a separately written checker re-derives every facet and cut from (H, s) in exact rationals, outside the decoder that produced the answer, and a tree of duals takes over where no single dual closes. Ott, Hetényi and Beverland [34] give decision-tree decoders for general quantum LDPC codes, one of which is guaranteed to return a minimum-weight correction, and apply them to this code; the guarantee is the algorithm's, not a certificate a third party checks. Aghababaie Beni, Higgott and Shutty [35] benchmark a search-based most-likely-error decoder on circuits

of this code. Safe dual bounds for linear programs solved in floating point go back to Neumaier and Shcherbina [9], whose bound is the lemma of Sec. II, and independently checkable branch-and-bound certificates are established in exact integer programming [36–38], so the branch-dual receipt is not new at that level of generality, and branch-and-bound over syndrome-decoding problems is the search of Ref. [8]; what is specific here is a checkable syndrome-decoding form, in which every node's bound is an exact dual the checker re-derives from (H, s), deployed behind a third-party heuristic decoder. Recent experiments decode with neural-network decoders fine-tuned on device data [39], and ensembling yields per-shot confidence [40]; neither is an optimality proof a third party can check.

Several recent preprints are close. Closest in setting is Banfi *et al.* [8], who certify per-shot outcomes on this code at three rounds, and on the [[72,12,6]] code, "under the full circuit-level noise model" of Ref. [5], as "a common reference for comparing decoders and codes". The two methods differ on four counts. *The objective.* They certify the outcome of non-degenerate maximum-likelihood decoding, whether the class of a minimum-cost pattern is the sampled error's class, by upper and lower bounds on the minimum cost in the true and in the competing classes, with costs $\log((1-q_j)/q_j)$ in nats, and they leave near-ties undecided; a receipt here fixes the minimum of a published integer objective for a syndrome, needs no sampled error, and certifies no class (Sec. VII). *Lower bounds and infeasibility.* Theirs come from disjoint-detector bounds, a charging bound whose linear program they "solve ... with HiGHS, recheck its constraints, and scale the charges down when necessary", best-first branch-and-bound, an adaptive parity-cut linear program of the Feldman and Taghavi–Siegel family used in Sec. V, and HiGHS mixed-integer dual bounds and zero-objective infeasibility tests; they add "exhaustive tests on small instances and independent solver comparisons", and on this code re-evaluated every certified failure with the exact integer-programming decoder bundled with Tesseract. Here every lower bound is a safe dual that the checker recomputes in exact rationals, and every infeasible leaf a GF(2) row combination it re-derives. *Floating point.* Their Appendix C.7 states that "costs, charges, and solver bounds use floating-point arithmetic", that "the current results therefore rely on the solver and its tolerances for numerical lower bounds and for the infeasibility claims", and that the margin ε "avoids verdicts near a tie, but does not bound every rounding error"; a class minimum counts as exact when the solver's dual bound and the incumbent agree to within 10^{-10}. Here floating point enters only the rounding that publishes the integer weights (Sec. II), the producer and its screen, and no acceptance depends on it. *What a third party receives.* They check every witness "against the detector and logical equations using integer arithmetic modulo two", a check that, in their words, "establishes feasibility, not optimality or the validity of the lower bound reported by a solver"; the paper describes no proof of the lower bounds that a third party can re-check without the solver (we did not inspect their code). A receipt here is such a proof, a file that two separately written checkers accept or refuse from (H, w, s) alone. Their verdict says more about logical outcomes; a receipt says less, exactly, in a form anyone can re-check; neither subsumes the other. Krishnamoorthy *et al.* [41] certify decoding decisions on bivariate-bicycle codes including this one; their target is the degenerate-coset maximum-likelihood class among candidates around the BP+OSD-0 answer, from which all their estimator-based decoders start, a different objective from the minimum single-error weight certified here. Their certificates are of three kinds: a paired bootstrap test on their sampler's estimates; an exact decision (their Lemma 1) when every class's partition function is estimated within a known constant factor, as the WISH estimator does with probability $1-\delta$, a route they validated with an exact optimization oracle on the distance-5 rotated surface code only (their Sec. 8.1) and whose realisation with a quantum annealer as that oracle they call "a concrete, if currently speculative, possibility"; and deterministic bounds from mini-bucket elimination (their Sec. 7.6), which at two bucket sizes certified 37 and 38 of 40 syndromes of the [[72,12,6]] code and none on this code, at 272 s per decode (their Sec. 8.5). None uses a linear program or a dual. On the [[72,12,6]] code their sampling decoder changes eight of BP+OSD-0's decisions for the better and three for the worse, across all rates (their Sec. 8.3), by changing the logical class, not by returning a certified minimum-weight correction. Shen and Zhong [42] test decoder libraries for conformance and release witnesses a standalone verifier re-derives, including a lighter feasible correction that proves a decoder's heavier answer non-minimal; their witnesses establish failures, whereas the receipts here establish optimality through an exact lower bound that excludes every smaller value on the weight lattice. Leverrier and Urbanke [43] give a decoder with circuit-level results on the same code that carries no certificate once pruned, and Sahay, Williamson and Brown [6] construct matching-based decoders for bivariate-bicycle codes. What is specific here is the combination: exact rational receipts, decided by a checker that imports no producer code and shares no arithmetic with it, wrapped around a third-party heuristic decoder rather than built into a decoder of our own, with branch-dual trees where the relaxation is fractional. We did not find a prior report of exact, independently re-checkable, minimum-weight receipt coverage for BP-OSD on this code; the nearest, Ref. [8], certifies another objective with bounds that rest on a solver. Our search, repeated for this revision, was logged but not exhaustive.

IX. REPRODUCIBILITY

Seeds, the frozen protocols of both campaigns, per-shot records, the persisted instances and integer weight vectors, every flat certificate and every tree are released with the code under the Apache License 2.0 at `https://github.com/coherence-energy-labs/oneq-certificates` (release v2026.11.07.2), with the companion manuscripts. The post-hoc analyses are released with their scripts: the scaled strong-baseline arms replay from their records and the stored proofs, the held-out re-decode reproduces every stored BP-OSD answer before it reports, and the receipt-defined tiers are re-decided from the stored proofs with the production exact checker. One command replays the complete receipt corpus against its pinned manifest with both checkers, no solver and no producer. A second, `tools/replay_qldpc_scaling_campaign.py`, replays the preregistered comparison of Sec. VI C from its archived inputs (`evidence/qldpc_scaling_prereg/inputs/`): for each dataset, the persisted check matrix by hash, the integer weight vector, the decoder's settings and channel, the logical test's stabilizer rows or observable columns, and every shot's sampled fault or observable value, bound to the record files by a manifest whose hash the tool pins. It re-checks all 6,500 receipts with both of their checkers, recomputes every answer's objective, score and logical outcome, and every count and McNemar p of Tables V and VI, and refuses on any mismatch; it needs Python's standard library and the checkers' own modules, and loads no producer, solver, decoder, sampler or NumPy. The archive was written once, by calling the runner's model-building code, and is accepted only where re-drawing the campaign's seeded samplers reproduced every recorded syndrome. A replay never re-samples: Stim's API reference for `compile_sampler` states that a seeded sampler's results are not consistent between Stim versions and may differ between machines with different SIMD widths or when the same shots are drawn in different batches, so no number here rests on re-sampling elsewhere. Receipt replay is complete from the release alone. The timed re-run of Sec. VI C needs the producer and the private development repository's history, not the archive. The public repository does not carry that history: the freeze, results and preregistration commits of Sec. VI are in the private repository, and the public one holds the files as released. Re-running the held-out sampling, decoding and production needs the code of the freeze commit: release v2026.09.16 of the public repository carries a snapshot of that revision, the producing scripts and every module they import, pinned by commit and sha256. The LP and MIP producers use SciPy 1.17.0 (HiGHS), BP-OSD is from `ldpc` 2.4.1, and sampling uses Stim 1.16.0. Every measured number in this paper is typeset from a macro generated from a JSON artifact, and a gate in the repository traces every number in the source, macro or literal, to the evidence tree before the manuscript builds. The generator rounds so as not to say more than the evidence: point percentages and lower bounds down at the last digit shown, upper bounds up, a range outward unless its ends are entries of a table it quotes, and every other value, timings and p-values included, to the nearest value at the digits shown.

Use of AI tools. AI tools assisted with software development and manuscript preparation. Reviews of this manuscript by AI agents that had not written it, and external audits of its drafts and earlier releases, informed it. Of the two latest audits, one calls itself AI-assisted and the other states that it was done by an AI model with AI sub-agents; the author's records describe the review of a draft that wrote the reference flat checker (Sec. IV) as AI-assisted; the others do not say who or what reviewed. Every review, audit and referee this paper mentions is one of these, and none is third-party peer review. The author is responsible for all content.

Appendix A: Verification log

This appendix keeps the provenance the main text states once.

The circuit dataset's shot rule. The predeclaration lists "150" for this rung beside nontrivial targets for the others; the runner took it as raw shots, and the 149 nontrivial are what is reported, a recorded deviation from the nontrivial rule and not a re-run. The second campaign states nontrivial targets for every dataset.

Producer tiers and receipt tiers. The held-out producer filed 1,780 shots as tier 1, 222 as tier 2 and 17 as trees, 8 of them a single dual; read by what the receipt proves, the eight single-dual trees are certified repairs, and the tier-1 count is unchanged because BP-OSD's answer was heavier on all eight. In the strong baseline the producer issued tier 1 only on shots with an integral primal; at $p \leq 0.03$ its counts equal the receipt-defined ones for every configuration, and at the two higher levels they are, of 200 shots each:

p	config.	tier 1, producer	tier 1, receipt	optimum hit
0.05	A	84	84	84
0.05	B	86	87	87
0.05	C	187	196	198
0.05	A_s	185	195	197
0.05	B_s	185	195	197
0.07	A	24	24	24
0.07	B	143	163	171
0.07	C	146	179	196
0.07	A_s	145	178	196
0.07	B_s	145	178	196

Stored after the fact. Four development trees, on the circuit dataset, were first stored after the campaign, bound to the answers the development records already held.

Regeneration. Regenerating the 2,002 held-out flat certificates from the shipped records gave 1,797 byte-identical to the run of record and 58 that differ while remaining exact and accepted by both checkers (147 circuit records carry no hash to compare); why those 58 choose a different optimal dual is not established. Repeated fresh processes, a warmed process that had priced the preceding shots, and several solver thread counts all reproduce today's certificate rather than the run of record's; an environment difference below the pinned package versions has not been excluded.

The JavaScript checkers. Writing them exposed no ambiguity in the specification; it exposed three rules of the harness that no stored receipt reached (no stored tree closes a leaf by infeasibility, sits on the lattice boundary or overloads a multiplier), which two deliberately broken copies of the tree checker passed until a hand-built instance whose one branch is provably infeasible was added, judged by the production checker as well. An external audit then found that both programs, having written their verdicts, exited before a piped standard output had drained, delivering 65,536 bytes of a larger batch with exit code 0; the harnesses caught it only because they reconcile receipt identifiers, and the programs now let the process end on its own.

The floating-point screen. One surviving mutant of the first mutation score exposed a defect in the screen, a path no receipt passes through: it admitted negative weights within its tolerance and could then prove a correction optimal while a lighter one existed. The screen now refuses every negative weight; the exact and tree checkers that decide every receipt always did.

The input domain. After the held-out campaign the checkers were changed to refuse syndrome entries other than 0 and 1, which they had reduced modulo 2. An external audit of the next release found the implementations still reading malformed input differently: the production checker truncated a check index `0.9` to `0` and accepted out-of-range check indices, the second tree checker read a syndrome entry `2` as `0`, and integer slots were coerced elsewhere. The contract of Sec. IV replaced all of these, with root guards in the two Python tree checkers to match the JavaScript one. No stored receipt was affected, since every integer of the stored corpus is a plain in-range integer. The final referee of this revision, an AI agent, then found that six of the seven implementations skipped a facet with multiplier 0 before validating it, so that a zero-multiplier facet naming check 999, or repeating another facet, was accepted unread; every implementation now validates every listed facet first, and the shared vectors include each facet rule on a zero-multiplier facet, flat and at tree depth 1. The stored corpora do list such facets, 983 of them in 121 receipts, and every one is well formed: all 2,042 receipts, the strong baseline's stored proofs and the 6,500 receipts of the second campaign verify under the changed bytes. Each change moved every pin of a changed checker with its previous value kept and its own record (`evidence/qldpc_receipt_reverification_2026-11-03.json`, then `evidence/qldpc_receipt_reverification_2026-11-03b.json`).

Bound-only results. Until the external audit, the bound-only mode of Sec. IV reported an exact optimum with a primal weight derived from T; no number in this paper read those fields.

The second campaign's deviation. Under the committed runner the six-round redundant-parity LP took 18.6, 42.9 and 206.5 s on three of the first 81 syndromes of the phenomenological dataset at $p = 0.02$, which did not finish in fifty minutes and wrote no record; a diagnosis on the same seeds printed timings only. The 4 datasets run before the change ran as committed; the 4 run after it carry their own code hashes in the summary, and of those only the phenomenological dataset at $p = 0.02$ uses the changed LP.

Post-hoc analyses. The scaled strong-baseline arms, the held-out re-decode of Table IV, the declared-rate scans and the receipt-defined tier recount were all run after the results they analyse were known, and are labelled post hoc where they appear. The comparison of Sec. VI C is the preregistered one.

[1] E. Berlekamp, R. McEliece, and H. van Tilborg, On the inherent intractability of certain coding problems (Corresp.), IEEE Transactions on Information Theory **24**, 384 (1978).

[2] M.-H. Hsieh and F. Le Gall, NP-hardness of decoding quantum error-correction codes, Physical Review A **83**, 052331 (2011).

[3] P. Panteleev and G. Kalachev, Degenerate quantum LDPC codes with good finite length performance, Quantum **5**, 585 (2021), arXiv:1904.02703.

[4] J. Roffe, D. R. White, S. Burton, and E. Campbell, Decoding across the quantum low-density parity-check code landscape, Physical Review Research **2**, 043423 (2020), arXiv:2005.07016.

[5] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, High-threshold and low-overhead fault-tolerant quantum memory, Nature **627**, 778 (2024).

[6] K. Sahay, D. J. Williamson, and B. J. Brown, A matching decoder for bivariate bicycle codes (2026), arXiv:2602.22770.

[7] J. Feldman, M. J. Wainwright, and D. R. Karger, Using linear programming to decode binary linear codes, IEEE Transactions on Information Theory **51**, 954 (2005).

[8] M. Banfi, M. Ferrari, A. Favano, A. Tarable, and L. Barletta, Certified non-degenerate maximum-likelihood benchmarking of quantum LDPC decoders (2026), arXiv:2610.28008.

[9] A. Neumaier and O. Shcherbina, Safe bounds in linear and mixed-integer linear programming, Mathematical Programming **99**, 283 (2004).

[10] X. Zhang and P. H. Siegel, Adaptive cut generation algorithm for improved linear programming decoding of binary linear codes, IEEE Transactions on Information Theory **58**, 6581 (2012).

[11] M. H. Taghavi and P. H. Siegel, Adaptive methods for linear programming decoding, IEEE Transactions on Information Theory **54**, 5396 (2008).

[12] M. H. Taghavi, A. Shokrollahi, and P. H. Siegel, Efficient implementation of linear programming decoding, IEEE Transactions on Information Theory **57**, 5960 (2011).

[13] T. Wadayama, Interior point decoding for linear vector channels based on convex optimization, IEEE Transactions on Information Theory **56**, 4905 (2010).

[14] C. Gidney, Stim: a fast stabilizer circuit simulator, Quantum **5**, 497 (2021), arXiv:2103.02202.

[15] C. Crocker, The gross code's syndrome circuit has circuit-level distance 10: an exhaustive lower bound (2026), doi:10.5281/zenodo.23452754.

[16] G. Pan, Interleaving recovers the full circuit distance of the gross and $[[90, 8, 10]]$ bivariate-bicycle codes, manuscript, code and data, release v2.0 of the GitHub repository gross-code-distance-12, `https://github.com/GavinNull/gross-code-distance-12`; first release v1.0 on 2026-10-20 (2026).

[17] G. Pan, Audit E-037: certificates of circuit-distance lower bounds for IBM's depth-8 circuits (gross [[144,12,12]], [[108,8,10]]), research record `research/audits/2026-10-26-E-037.md` on the main branch of `https://github.com/GavinNull/gross-code-distance-12`, commit 34ffcb14 of 2026-10-26 05:39 (UTC+8), after release v2.0 (2026), title translated from the Chinese heading.

[18] G. Pan, Audit E-043: completing table I; certification for every R of IBM's two depth-8 circuits in $T(8)$, and three new circuits, research record `research/audits/2026-10-31-E-043.md` on the main branch of `https://github.com/GavinNull/gross-code-distance-12`, commit 18e47dae of 2026-10-31 16:55 (UTC+8), after release v2.0 (2026), title translated from the Chinese heading.

[19] G. Pan, Audit E-045: certificates for the Tesseract CZ-compiled gross circuit, every R for IBM's 12+2 model, and the monotonicity of d_{circ} in R, research record `research/audits/2026-11-04-E-045.md` on the main branch of `https://github.com/GavinNull/gross-code-distance-12`, commit 488c1839 of 2026-11-04 14:50 (UTC+8), after release v2.0 (2026), title translated from the Chinese heading.

[20] B. Cheng, N. Guo, and F. Pan, Certifying quantum code and circuit distances at scale (2026), arXiv:2611.08247.

[21] M. Ehatamm, Y. Lee, X. Wu, and R. Tao, End-to-end formalization of quantum error correction (2026), arXiv:2605.16523.

[22] D. Poulin and Y. Chung, On the iterative decoding of sparse quantum codes, Quantum Information and Computation **8**, 987 (2008), arXiv:0801.1241.

[23] G. Fregona, M. C. Löbl, B. J. Brown, M. J. Kastoryano, and S. Paesani, Dealing with low-weight errors that lead BP OSD to fail (2026), arXiv:2610.30305.

[24] R. M. McConnell, K. Mehlhorn, S. Näher, and P. Schweitzer, Certifying algorithms, Computer Science Review **5**, 119 (2011).

[25] M. Blum and S. Kannan, Designing programs that check their work, Journal of the ACM **42**, 269 (1995).

[26] J. X. Li and P. O. Vontobel, LP decoding of quantum stabilizer codes, in *2018 IEEE International Symposium on Information Theory (ISIT)* (2018) pp. 1306–1310.

[27] N. Halabi and G. Even, Local optimality certificates for LP decoding of Tanner codes (2011), arXiv:1104.4646.

[28] A. J. Landahl, J. T. Anderson, and P. R. Rice, Fault-tolerant quantum computing with color codes (2011), arXiv:1108.5738.

[29] S. C. Draper, J. S. Yedidia, and Y. Wang, ML decoding via mixed-integer adaptive linear programming, in *2007 IEEE International Symposium on Information Theory (ISIT)* (2007) pp. 1656–1660.

[30] S. Javed, F. Garcia-Herrero, B. Vasić, and M. F. Flanagan, Low-complexity linear programming based decoding of quantum LDPC codes, in *ICC 2024 – IEEE International Conference on Communications* (2024) pp. 1782–1787, arXiv:2311.18488.

[31] S. Gu and M. Soleimanifar, Power and limitations of linear programming decoder for quantum LDPC codes, IEEE Transactions on Information Theory **72**, 7604 (2026), arXiv:2508.04769.

[32] Y. Wu, B. Li, K. Chang, S. Puri, and L. Zhong, Minimum-weight parity factor decoder for quantum error correction (2025), arXiv:2508.04969.

[33] J. Hwang, B.-S. Won, and U. Hwang, Certified adaptive list decoding for error-correcting codes (2026), arXiv:2610.27805.

[34] K. R. Ott, B. Hetényi, and M. E. Beverland, Decision-tree decoders for general quantum LDPC codes (2025), arXiv:2502.16408.

[35] L. Aghababaie Beni, O. Higgott, and N. Shutty, Tesseract: A search-based decoder for quantum error correction (2025), arXiv:2503.10988.

[36] K. K. H. Cheung, A. Gleixner, and D. E. Steffy, Verifying integer programming results, in *Integer Programming and Combinatorial Optimization*, Lecture Notes in Computer Science, Vol. 10328 (2017) pp. 148–160.

[37] L. Eifler and A. Gleixner, A computational status update for exact rational mixed integer programming, Mathematical Programming **197**, 793 (2023).

[38] D. E. Steffy and K. Wolter, Valid linear programming bounds for exact mixed-integer programming, INFORMS Journal on Computing **25**, 271 (2013).

[39] Google Quantum AI and Collaborators, Quantum error correction below the surface code threshold, Nature **638**, 920 (2025), arXiv:2408.13687.

[40] N. Shutty, M. Newman, and B. Villalonga, Efficient near-optimal decoding of the surface code through ensembling, Physical Review Letters **136**, 070603 (2026), arXiv:2401.12434.

[41] R. Krishnamoorthy, F. Gerhardt, J. Knaute, T. Klir, S. R. Maschek, E. Schulze, T. Maras, A. Dotterweich, L. K. Lee, C. Bauckhage, and N. Piatkowski, Certified decoding of quantum LDPC codes (2026), arXiv:2608.25545.

[42] J. Shen and H. Zhong, Capability-gated conformance testing of quantum error-correction decoder libraries (2026), arXiv:2609.07035.

[43] A. Leverrier and R. Urbanke, Approximating optimal decoding of quantum LDPC codes with narrow frontiers (2026), arXiv:2606.20513.